

CSE 341 — Concepts of Programming Languages

WorkoutLang — D4: AI Usage Journal (Part 1)

Gebze Technical University | Spring 2026 | Part 1 entries: 30 Apr – 7 May 2026

D4 — AI Usage Journal

This journal documents my substantive interactions with AI assistants during Part 1 of the WorkoutLang project. Each entry records the goal, the exact prompt sent, the relevant portion of the response, what I accepted or rejected, any errors I caught, and a reflection on what the session taught me. Entries are ordered chronologically and dated on the day the session actually took place.

Entry 1 of 5

Entry #	1 of 5
Date	2026-04-30
Phase	Design — domain selection and language overview
AI tool	Claude Sonnet 4.6
Goal	I had the fitness idea in mind but wasn't sure it was interesting enough, and I wanted to know what constructs the language would need.
Prompt	Verbatim prompt sent: <code>is fitness a good DSL topic? I want to make a workout programming language, what kind of constructs should it have</code>
Response (key part)	It gave a decent list: a set statement (exercise name, sets×reps, rest duration), workout/routine blocks, a load keyword, repeat N weeks as a loop, and duration/weight token types with physical suffixes. It suggested Day as the structured type. Then it said “dynamic typing is more flexible.” I asked “is this domain complex enough” and it said “borderline, but the set statement works as a domain-specific construct.” I asked a follow-up: “Is this domain complex enough for the project? The handout says the language should have interesting design questions.” The AI said it was borderline — the set statement gives a genuine domain-specific construct, but without weight-on-exercises or progression rules it might feel thin. That pushed me to keep the weight type even though I wasn't sure I'd use it.
Accepted	The set statement shape (exercise reps N x M rest D) and the workout/routine/func split. The DURATION_LIT idea was useful too — I had been thinking of parsing 90s as INT + IDENT, making it a single token is cleaner.
Rejected / Modified	Dropped Day as a record type — looking at the handout's “interesting design questions” phrase, Day has nothing interesting, just a name and some sets. I wanted to drop dynamic typing too but honestly at that point I didn't fully understand what strong typing would mean for my types, so I just went with “the handout wants a type system, don't touch it.”
Errors caught	The AI said “repeat N weeks is basically a for loop.” This sent me down the wrong path — while writing the grammar I tried to add a counter variable inside repeat, then realized that's not the language I designed. repeat has no counter, it just says how many times to run.
Reflection	Good as a starting point but my question was too broad so I got a broad answer. “What constructs should a workout language have” is not a great question. “What type design decisions would be interesting in a workout DSL” would have been more useful.

Entry 2 of 5 [Experiment E1 — EBNF Generation]

Entry #	2 of 5 — Experiment E1
Date	2026-05-01
Phase	Design — EBNF grammar
AI tool	Claude Sonnet 4.6
Goal	I had a rough grammar draft. The handout said Sebesta EBNF and I wasn't sure I was using the right format. Wanted the AI to generate one from scratch so I could compare.
Prompt	Verbatim prompt sent: <pre> WorkoutLang is a DSL for fitness programming. A program consists of workout declarations, routine declarations, and function declarations. A workout contains day blocks, each of which contains set statements of the form: set EXERCISE reps N x M rest DURATION. A routine contains if/else, while, repeat N weeks, load, print, return, variable declarations, and assignments. Functions have typed parameters and a return type. Types are: int, float, bool, duration, weight. Write a complete EBNF grammar for this language in Sebesta style. </pre>
Response (key part)	The AI produced a grammar with the following structure (excerpt): <pre> <program> ::= { <decl> } <decl> ::= <workout_decl> <routine_decl> <func_decl> <set_stmt> ::= "set" IDENT "reps" INT "x" INT "rest" DURATION <repeat_stmt> ::= "repeat" INT "weeks" <block> <expr> ::= <expr> ("+" "-") <term> <term> <term> ::= <term> ("*" " /") <factor> <factor> </pre> The top level looked fine — <program>, <workout_decl>, <set_stmt> all there and matched mine. Then I saw <expr> ::= <expr> ("+" "-") <term> <term>. I thought “that’s just normal grammar notation” and moved on. But something nagged at me. Looked at Sebesta — he uses { } for this, different.
Accepted	Good as a cross-check. I had forgotten the return type rule in my draft — I only noticed because it was in the AI’s version.
Rejected / Modified	I didn’t see the problem in the expression rule at first — “it’s left-recursive, that’s normal.” When I started writing the parser the function was calling itself forever. I asked the AI “does your grammar cause infinite recursion in a recursive descent parser” and it said yes, left-recursive rules don’t work top-down, Sebesta uses { } specifically for this. The == and < being at the same level I also skipped thinking “does it matter.” Later I wrote something like 1 == 2 < 3 and the parser produced the wrong tree — separate levels needed. The x-as-IDENT thing I nearly missed — I declared a variable named x inside a test routine, wrote set x reps 3 x 10, and the lexer got confused with two different x tokens.

Errors caught	I initially assumed my parser was wrong and spent about 20 minutes looking at my own code. The grammar was the problem. The x conflict I found by accident while writing tests.
Reflection	I almost copied the grammar straight in. The left-recursion bug would not have shown up until the parser hung at runtime. Now whenever I see an expression rule I first check whether it uses { } and then test something like 4 - 2 - 1.

Entry 3 of 5

Entry #	3 of 5
Date	2026-05-03
Phase	Lexer implementation
AI tool	Claude Sonnet 4.6
Goal	I wanted 90s to be a single token but I started by asking the wrong thing — I was thinking about whether to use if/else for keywords and got sidetracked.
Prompt	Verbatim prompt sent: <pre>should I use if/else to check for keywords in the lexer or is there a better way? also how do I recognize things like 90s as a token</pre>
Response (key part)	The AI suggested: after reading all digit characters into a buffer, peek at the next character(s) to detect the suffix. For 's', check that the character after it is not alphanumeric (to avoid matching 'seconds' as a duration). For 'min' and 'hr', check two or three characters ahead. If a suffix is detected, append it to the buffer and advance past it, then return the appropriate token type. It also gave me a peek_ahead(n) helper function to look n characters ahead. Most of the code was right but there was one thing I nearly missed.
Accepted	The lookup table for keywords and the core suffix approach. The !isalnum(peek1) check was the important part — without it the lexer would try to match 's' in the middle of a word like 'sets'.
Rejected / Modified	I copied the peek_ahead(n) function, then realized l->src[l->pos + 1] already does the same thing. Unnecessary, deleted it. It also suggested 'sec' as an alias for 's'. Didn't add it — it's not in the token table, not in the spec.
Errors caught	The buffer in the AI's code was 32 bytes; I already had char lexeme[256] in token.h. Got a compile error after copying because the signature didn't match. Asked why 32, it said "just kept it short for the example." Not intentional, just sample code.
Reflection	My question started as "should I use if/else for keywords" which was the wrong thing to ask — of course not, a lookup table is obvious. The real question was "how do I recognize number + suffix as a single token." If I had thought about it for five minutes first I would have figured out the peek approach myself. Also, giving the AI my existing struct upfront would have stopped it generating wrappers I didn't need.

Entry 4 of 5

Entry #	4 of 5
Date	2026-05-07
Phase	Writing — D1 Section 4.6 (Names, Binding, Scope, Lifetime)
AI tool	Claude Sonnet 4.6
Goal	I was curious which was better, static or dynamic scoping, and wanted to ask the AI. I also wanted to check my §4.6 draft for terminology.
Prompt	Verbatim prompt sent: <pre>which is better, static scoping or dynamic scoping? which one should I use</pre> [follow-up: pasted my §4.6 draft and asked "is this terminology correct"]
Response (key part)	For the scoping question it said "static is generally better because it's more readable" and mentioned dynamic scoping is used in early LISP dialects. For the §4.6 draft the useful thing was: I had written "variables are bound at declaration time" and the AI said this was incomplete. Sebesta §5.3 separates name-to-type binding (compile time) from name-to-value binding (run time) — they're different events. It also brought up a third category: load-time bindings.
Accepted	The compile-time vs run-time binding split was a real gap — fixed that. Added the section numbers too.
Rejected / Modified	The load-time bindings category. I looked it up in Sebesta — it only applies to languages with a separate compilation and linking step. WorkoutLang doesn't have one. The AI presented it as if it were always relevant.
Errors caught	The AI said "dynamic scoping is used in early LISP dialects." I wanted to put this in the spec but I vaguely remembered Common Lisp being lexically scoped. Checked Sebesta: LISP 1.5 and some Perl scoping modes are mentioned as dynamic. Common Lisp is lexical by default. Not completely wrong but not precise enough — I would have said something incorrect in the exam.
Reflection	Asking "which one should I use" got me a shallow answer — "static is generally better" is not something I can defend in an exam. Getting the AI to review my own draft was more useful. Without the habit of checking the book I would have put the LISP claim in the spec.

Entry 5 of 5

Entry #	5 of 5
Date	2026-05-07
Phase	Parser — AST node design in C
AI tool	Claude Sonnet 4.6
Goal	I needed to write ast.h before I could start the parser. My first question to the AI was way too vague.
Prompt	Verbatim prompt sent: <code>how do I write ast.h</code>
Response (key part)	It dumped a long response — two approaches, tagged union (ExprKind enum + C union) or void* with function pointers. Skeletons for both. Recommended tagged union for simplicity. I asked a follow-up: “which one is easier to write a printer for” because the AST printer is a required deliverable. Tagged union, it said — just switch on kind and handle each case. Copied it and tried to compile.
Accepted	The tagged union structure and the EXPR_INT, EXPR_BINOP naming. Used them as-is. The undefined behaviour warning was useful — made me add default: assert(0) to every switch in ast.c.
Rejected / modified	The AI put char op[4] at the top level of Expr, outside the union — so an IntLit node would also have an op field. That makes no sense. Moved it inside the binop sub-struct. There was also an ASTNode typedef aliasing Expr*, I asked what it was for, it said “style preference.” Dropped it — cleaner without it.
Errors you caught	No line field anywhere in the skeleton. I only noticed while writing the parser — I had no way to report which line an error was on. Asked the AI “the type checker needs to know what line a type error is at, where does that go,” it said “add int line to each node struct.” It hadn’t mentioned this on its own.
Reflection (2–3 sentences)	“How do I write ast.h” is a terrible question — I got a long skeleton and had to dig through it for what I actually needed. If I had asked “the type checker in Part 2 will consume these structs, what fields does it need” the line field would have come up immediately.